MODULE 01 · DOMAIN 1 - MONITORING, LOGGING, ANALYSIS, REMEDIATION, AND PERFORMANCE OPTIMIZATION

The CloudWatch Metric Model

Namespaces, dimensions, resolution and statistics. Get this wrong and every alarm you build is wrong quietly.

DOMAIN 1 of 5 EXAM WEIGHT 22% MODULE 1 of 35

admin@sthithapragna.com

A metric is not a name. It is a **coordinate**. Change any part of it and you have a different metric, not a different reading.

What identifies a metric

- **Namespace.** The container. AWS services use **AWS/EC2**, **AWS/RDS**, **AWS/Lambda**. Yours cannot start with AWS slash.
- **Metric name.** CPUUtilization, Duration, FreeStorageSpace.
- **Dimensions.** Up to **30** name and value pairs. They are part of the identity, not a filter applied afterwards.
- **Region.** Metrics never leave the Region they were published in.

> ON THE EXAM
>
> Namespace plus name plus the exact set of dimensions. All four must match or it is a different metric.

Why dimensions catch people out

- **You cannot aggregate across a dimension you invented.** If a service does not publish a rolled-up version, there is nothing to graph.
- Adding one dimension to an existing custom metric **starts a new series from zero**. The old history stays where it was.
- AWS services publish the **same metric at several dimension combinations** deliberately, so both views exist.
- High-cardinality dimensions such as request ID or customer ID **multiply your bill**, because each combination is a billable custom metric.

> ON THE EXAM
>
> Design dimensions for the questions you will ask. Retrofitting one splits your history in two.

EC2 metrics are collected from **outside** the instance. That is why there is no memory metric.

In the box by default, and not

MEASUREMENT	AVAILABLE WITHOUT AN AGENT?
CPUUtilization	Yes. Hypervisor-visible
NetworkIn / NetworkOut	Yes. Counted on the virtual NIC
EBS read and write ops	Yes. Counted at the volume
StatusCheckFailed	Yes. Instance, system and attached EBS checks
Memory used, available	NO. Needs the CloudWatch agent
Disk space used on a filesystem	NO. Needs the CloudWatch agent
Page file or swap usage	NO. Needs the CloudWatch agent

ON THE EXAM

If the question says memory or disk space, the answer involves the CloudWatch agent. Every time.

Basic or detailed monitoring?

BASIC - THE DEFAULT

Five-minute periods

- Free, on by default for EC2
- Data points every 300 seconds
- Fine for capacity trends and slow-moving alarms
- An alarm cannot react faster than the data arrives

DETAILED - OPT IN

One-minute periods

- Charged per instance per month
- Data points every 60 seconds
- Needed for responsive scaling and incident response
- Enable per instance, or in the launch template

THE TIEBREAKER

An alarm evaluating a 60-second period against a metric published every 300 seconds spends most of its life in INSUFFICIENT_DATA.

Two related traps

- **Auto Scaling group metrics are off by default** and are separate from instance metrics. GroupInServiceInstances and friends must be enabled.
- **ASG group metrics are always one-minute** and free. Detailed monitoring on the instances is a different setting.
- **Detailed monitoring is set in the launch template** for scaled instances. Setting it on one instance by hand does not survive a scale event.
- Metrics with no new data points for **two weeks stop appearing** in the console. The data is still there for the full retention.

> ON THE EXAM
>
> "The dashboard is empty for the auto scaling group" is almost always disabled group metrics, not a permissions problem.

Standard or high resolution?

STANDARD RESOLUTION

One-minute granularity

- Everything AWS publishes, always
- Alarm periods are multiples of 60 seconds
- The default for your custom metrics

HIGH RESOLUTION

One-second granularity

- Custom metrics only, StorageResolution set to 1
- Alarm periods of 10 or 30 seconds become available
- Retrievable at 1, 5, 10, 30 seconds or multiples of 60
- Costs more, and only survives 3 hours

THE TIEBREAKER

High resolution is a property of the data you publish. You cannot make an AWS service metric high resolution.

Retention, and what rollup actually does

PERIOD OF THE DATA POINT	HOW LONG YOU CAN RETRIEVE IT
Under 60 seconds (high resolution)	3 hours
60 seconds (1 minute)	15 days
300 seconds (5 minutes)	63 days
3600 seconds (1 hour)	455 days, about 15 months

ON THE EXAM

Nothing is deleted at 15 days. It is aggregated. Your data survives 15 months, just not at the resolution you wrote it.

Timestamps and the awkward edges

- A data point's timestamp can be **up to two weeks in the past and two hours in the future**. Outside that it is rejected.
- Backfilling a metric older than two weeks is **not possible**. This kills a lot of migration plans.
- Omit the timestamp and CloudWatch stamps it on receipt, which hides clock skew on the sender.
- CloudWatch aggregates to a floor of **one second** even if you send finer timestamps.

> ON THE EXAM
>
> "We want to import last quarter's metrics into CloudWatch" cannot be done. The two-week window is hard.

Statistics: the number you choose changes the answer

- **Average, Sum, Minimum, Maximum, SampleCount** are the basic five.
- **Percentiles** such as p90, p95, p99 need raw data points, so they do not work on pre-aggregated statistic sets.
- Percentiles are **unavailable if any value is negative**.
- **Trimmed mean** and trimmed count discard outliers at the ends, useful when a few extreme values distort the mean.

> ON THE EXAM
>
> The statistic is chosen at read time, not at write time. The same stored data answers all of them.

Average or percentile?

AVERAGE TELLS YOU

How the system is doing

- Good for capacity and cost
- Good for CPU-driven scaling
- Hides a slow minority completely
- One request in a hundred taking 30 seconds barely moves it

P99 TELLS YOU

How your worst users are doing

- Good for latency and user experience
- Good for SLA and SLO alarms
- Catches the tail the average buries
- Noisy on low-traffic metrics with few samples

THE TIEBREAKER

The average customer experience is nobody's experience. If a scenario mentions some users, complaints, or an SLA, it wants a percentile.

Querying metrics: three tools, three jobs

- **Metric math** combines existing metrics: rates, sums across resources, error percentages. Alarms can watch a math expression.
- **Search expressions** match metrics by pattern, so a dashboard picks up new instances **without being edited**.
- **Metrics Insights** is a SQL-like query over metrics, with GROUP BY and ORDER BY, for high-cardinality exploration.
- An alarm on a search expression **needs metric math to collapse it** to a single series first.

> ON THE EXAM
>
> "The dashboard must include instances added later" is a search expression. Nothing else does that.

Publishing your own: two doors

PUTMETRICDATA

An explicit API call

- Direct, synchronous, easy to reason about
- Every call is charged
- Adds latency to the code path
- Fine for a few metrics on a schedule

EMBEDDED METRIC FORMAT

Structured JSON in a log line

- Write a log line, CloudWatch extracts metrics from it
- No API call, no added latency
- Keeps the full log context beside the metric
- The recommended path from Lambda

THE TIEBREAKER

High volume, or inside a Lambda function, or you also want the log line: EMF. Otherwise PutMetricData is fine.

One pane of glass: cross-account observability

- One **monitoring account** creates a **sink**. Each **source account** creates a **link** to it.
- Shares **metrics, logs and traces**. Search, graph, run Logs Insights and alarm across accounts.
- **No extra charge** for logs and metrics. Works with Organizations for bulk onboarding.
- It is **within a Region**. For cross-Region use centralization rules or a cross-account metric stream.

> ON THE EXAM
>
> Sink in the monitoring account, link in each source account. Getting the direction backwards is a classic distractor.

Getting metrics out: metric streams

- **Metric streams** push metrics continuously to **Amazon Data Firehose**, in near real time.
- From Firehose to S3 for a data lake, or to a third-party tool such as a partner monitoring product.
- The alternative is polling **GetMetricData** on a schedule, which is slower and costs API calls.
- Streams can be filtered by namespace, and can be **cross-account** from a monitoring account.

> ON THE EXAM
>
> "Continuously export CloudWatch metrics to a third party or a data lake" is a metric stream, not a Lambda on a schedule.

A CloudOps engineer runs a fleet of Linux web servers in an Auto Scaling group, and the instances are replaced regularly. An alarm is needed whenever any instance has less than 10 percent free space on its /var partition.

The engineer opens the CloudWatch console, filters to the AWS/EC2 namespace, and cannot find a disk space metric for any instance.

What should the engineer do?

A Enable detailed monitoring on the Auto Scaling group's launch template

B Install and configure the CloudWatch agent through the launch template's user data, publishing disk_used_percent

C Enable Auto Scaling group metrics collection on the group

D Create a metric math expression deriving free space from EBS read and write operations

◆ PAUSE AND DECIDE

What should the engineer do?

A. Enable detailed monitoring

Changes how often existing metrics arrive, from five minutes to one. It cannot create a metric that does not exist.

B. CloudWatch agent via the launch template

Filesystem usage is only visible inside the guest, so an agent is the only source. Putting it in the launch template means replaced instances still report.

C. Enable group metrics

Gives you instance counts by lifecycle state for the group. Says nothing about anything inside an instance.

D. Metric math from EBS ops

Read and write operations count activity, not occupancy. A full volume and an empty one can produce identical numbers.

THE SIGNAL

Two clauses decide it: disk space means agent, and replaced regularly means the launch template rather than a one-off install.

> An application publishes a custom metric named CheckoutLatency in the namespace Shop, with a single dimension of Environment set to production. A dashboard has graphed it for a year.
>
> A release adds a second dimension, Region, to every published data point. The next morning the dashboard shows a line that stops at the release time, and the on-call engineer reports that the latency alarm has been in INSUFFICIENT_DATA since the deployment.

Why did this happen?

A The new dimension exceeded the 30-dimension limit, so the publish calls are failing

B Adding a dimension creates a new metric, and the old dimension combination is no longer receiving data

C Custom metrics are retained for 15 days, and the historical data has aged out

D The metric became high resolution, so it is only retained for three hours

◆ PAUSE AND DECIDE

Why did this happen?

A. Exceeded the 30-dimension limit

Two dimensions is nowhere near thirty. And a rejected publish would show as errors in the application, not a tidy line ending.

B. The dimension set changed the metric's identity

Namespace plus name plus the exact dimension set is the identity. Environment alone and environment plus region are two different metrics.

C. Retention aged the data out

Fifteen days applies to one-minute resolution and then rolls up rather than deleting. It also would not coincide with a release.

D. It became high resolution

High resolution is set by StorageResolution, not by dimensions, and it would not affect the old series at all.

THE SIGNAL

The alarm still watches Environment=production alone. That coordinate exists and is empty, which is what INSUFFICIENT_DATA means.

A payments API is meeting its published target of a 250 millisecond response time when measured as the average over one-minute periods. Support is nonetheless receiving daily complaints from a small number of merchants about requests that hang for many seconds before completing.

The team wants an alarm that fires on the behaviour the merchants are describing, using the existing latency metric.

What should the team configure?

A. Change the alarm to the Maximum statistic over the same period

B. Change the alarm to the p99 statistic and set a threshold against the tail

C. Enable high resolution on the metric so slow requests are not averaged away

D. Shorten the alarm period to 10 seconds so short spikes are visible

◆ PAUSE AND DECIDE

What should the team configure?

A Maximum statistic

The genuinely tempting one. It does surface slow requests, but a single outlier fires it, so it alarms constantly and gets muted.

B p99 statistic

Measures the experience of the worst one per cent as a population, so it tracks a real degradation and ignores a lone straggler.

C High resolution

Changes how often data points exist, not how they are summarised. Averaging one-second data still buries the tail.

D 10-second period

Also a frequency change, and it needs a high-resolution metric to be legal at all. The average is still an average.

THE SIGNAL

A small number of users complaining while the average looks fine is the textbook percentile signal. Resolution and statistic are different axes.

A company runs 40 accounts under AWS Organizations. The central operations team must graph metrics, search log groups and alarm on resources in every account from one place, without engineers switching roles to investigate.

The team also needs the same metrics delivered continuously into an existing third-party monitoring product that the company is contractually committed to for another two years.

Which combination should the team implement?

A. Cross-account observability with a sink in the operations account, plus a metric stream to Amazon Data Firehose

B. A scheduled Lambda function in each account calling GetMetricData and writing to a central S3 bucket

C. Cross-account observability with a sink in each source account, plus CloudWatch dashboards shared by URL

D. An organization CloudTrail trail, with the third party reading the trail's S3 bucket

◆ PAUSE AND DECIDE

Which combination should the team implement?

A Sink in operations, plus a metric stream

Correct direction for the sink, and a stream is the managed continuous export. Both halves of the requirement, no code.

B Scheduled Lambda per account

Forty functions to maintain, polling rather than streaming, and it delivers files to a bucket rather than a searchable console view.

C Sink in each source account

The direction is backwards. One sink lives in the monitoring account; sources create links. Shared URLs also do not give you cross-account alarms.

D Organization CloudTrail trail

CloudTrail records API calls, not metrics. It answers who did what, which is not what either requirement asked for.

THE SIGNAL

Two requirements, two mechanisms. "From one place" is cross-account observability; "continuously, to a third party" is a metric stream.

The five things to carry forward

- **A metric is a coordinate:** namespace, name, exact dimension set, Region. Change one and it is a different metric.
- **Memory and disk space need the agent.** The hypervisor cannot see inside the guest, and no setting changes that.
- **Resolution and statistic are different axes.** Frequency of data versus how it is summarised.
- **Retention rolls up, it does not delete.** 3 hours, 15 days, 63 days, 455 days.
- **Sink in the monitoring account, links in the sources.** Streams push out to Firehose.

> ON THE EXAM
>
> Next: alarms. Everything here is the input to them, which is why they are in this order.